

Trabalho Prático 3: Sistema de E-commerce com Consultas Multiatributo

Gustavo Barra Felizardo
Matrícula: 2025051020
DCC/UFMG

DCC221 – Estruturas de Dados – 2026/1

1 Introdução

Plataformas de comércio eletrônico mantêm um grande volume de entidades, como usuários, produtos, compras e reposições, e precisam responder a consultas que combinam vários atributos sobre elas. O desafio não está em armazenar os dados, e sim em recuperá-los com eficiência. Uma busca ingênua, que varre todas as entidades a cada consulta, fica inviável conforme a base cresce.

Este trabalho implementa o MercadoDCC, um sistema que resolve esse problema por meio de **índices invertidos**. Para cada atributo buscável, mantém-se uma estrutura que associa cada valor ao conjunto de identificadores das entidades que o possuem, uma consulta por múltiplos atributos é então resolvida combinando esses conjuntos por interseção. Como o uso de contêineres da biblioteca-padrão é proibido pelo enunciado, as estruturas de suporte (vetor dinâmico, tabela hash e árvore AVL) foram implementadas do zero.

O sistema oferece oito operações. Quatro registram dados: **U** cadastra um usuário, **P** cadastra um produto, **R** repõe estoque e **C** registra uma compra, efetivada apenas se houver estoque para todos os itens. As outras quatro consultam por combinação de atributos: **LU** sobre usuários, **LP** sobre produtos, **LC** sobre compras e **LR** sobre reposições. As consultas de usuários e produtos ainda imprimem, quando existem, os produtos comprados e os respectivos compradores.

O restante do relatório descreve a solução (Seção 2), analisa sua complexidade (Seção 3), discute a robustez (Seção 4) e avalia experimentalmente o desempenho (Seção 5).

2 Método

2.1 Tipos Abstratos de Dados

Os quatro TADs obrigatórios foram definidos em arquivos próprios. **Usuario** guarda id, nome, idade, cidade, estado e nacionalidade. **Produto** guarda id, nome, preço, quantidade em estoque, categoria, marca e condição. **Compra** guarda id, timestamp, id do usuário e a lista de itens comprados, **Reposicao** guarda id, timestamp e a lista de itens repostos. Cada item é um par (id do produto, quantidade). Os identificadores são atribuídos auto-

maticamente em ordem crescente a partir de zero, o que permite armazenar as entidades em vetores indexados pelo próprio id, com acesso direto $O(1)$.

2.2 Estruturas de dados genéricas

Três estruturas genéricas (templates) sustentam todo o sistema:

- **Vetor**: bloco contíguo de tamanho variável que dobra de capacidade na realocação, garantindo inserção ao final amortizada $O(1)$. Armazena as entidades, as posting lists e os históricos agregados.
- **Hash**: tabela hash com encadeamento separado, função djb2 (uma função clássica de hashing de strings) e redimensionamento por fator de carga. Indexa os atributos textuais (nome, cidade, estado, nacionalidade, categoria, marca, condição).
- **AVL**: árvore binária de busca balanceada por rotações. Indexa os atributos numéricos (idade, timestamp) e, por manter as chaves ordenadas, suporta consulta por faixa.

2.3 Índices invertidos e estruturas auxiliares

O valor de cada índice é uma posting list, um **Vetor** de ids. Como os ids são atribuídos em ordem e inseridos no cadastro, cada lista é construída por append, nascendo ordenada, isso torna a interseção e a união simples operações de merge linear, e dispensa ordenação na saída. Atributos cuja chave já é um id contíguo (`id`, `id_usuario`, `id_produto`) usam vetores indexados diretos, sem hash nem AVL. Para a segunda linha das consultas LU e LP, mantêm-se históricos agregados: para cada usuário, os produtos comprados com as quantidades totais, e o simétrico para cada produto. Conceitualmente, cada índice mapeia um valor a um conjunto de ids, por exemplo `idade` $\rightarrow 21 \rightarrow \{0, 3, 7\}$.

2.4 Operações e métodos

As oito operações são despachadas pelo primeiro token de cada linha. A compra (`C`) valida o estoque de todos os produtos antes de efetivar, se insuficiente, não altera o estado e produz `C INV`, garantindo atomicidade. As consultas avaliam os filtros e imprimem os resultados em ordem crescente de id. Cada operação tem seu próprio método (`cmdU`, `cmdP`, `cmdR`, `cmdC`, `cmdLU`, `cmdLP`, `cmdLC`, `cmdLR`), todos despachados pelo método `processar` conforme o primeiro token da linha.

2.5 Avaliação das consultas

Toda consulta passa por duas etapas. Primeiro, cada filtro atômico, um par atributo e valor, é resolvido em sua posting list pelo índice correspondente: a hash devolve a lista do valor textual, a AVL devolve a lista da chave numérica e os atributos que já são id usam o vetor direto. Em seguida, as listas são combinadas. No formato principal, sem operadores, todos os filtros são unidos por interseção, percorrendo as listas ordenadas em paralelo e mantendo apenas os ids presentes em todas. O método `avaliarConsulta` inspeciona os tokens: se encontra `AND`, `OR` ou `NOT`, encaminha para o analisador booleano (`parseOu`, `parseE`, `parseNao` e `parseAtomo`, um método por nível de precedência), caso contrário aplica a interseção direta.

2.6 Decisões de projeto

A escolha de cada estrutura seguiu o tipo de atributo. Os atributos textuais, como cidade e marca, não têm ordem natural útil e admitem muitos valores distintos, então a tabela hash, com busca $O(1)$ em média, é a opção mais direta. Os atributos numéricos, idade e timestamp, precisam, além da busca exata, responder a consultas por faixa, o que exige manter as chaves ordenadas, e a AVL atende aos dois casos com altura $O(\log A)$. Os atributos que já são identificadores contíguos dispensam qualquer índice associativo, pois o próprio id serve de posição em um vetor. Essa organização concentra o custo no que de fato varia com a carga, o tamanho das posting lists, e mantém o cadastro barato.

2.7 Pontos extras

Um único executável trata os dois formatos de consulta. Para os **operadores booleanos**, um analisador de descida recursiva implementa a precedência NOT > AND > OR (um nível de recursão por operador), combinando as posting lists por interseção, união e complemento em relação ao universo da entidade. A interseção e a união são percursos simultâneos (merge) de duas posting lists ordenadas, com custo $O(|A| + |B|)$, e o complemento percorre o universo, com custo $O(N)$. Para as **faixas numéricas**, detectadas quando um atributo numérico é seguido de dois valores, a AVL recupera por um percurso em ordem podado as listas cujas chaves caem no intervalo, que são então unidas.

3 Análise de Complexidade

A Tabela 1 resume a complexidade de tempo. A notação: n, m, c, r são os números de usuários, produtos, compras e reposições, A é o número de valores distintos do atributo indexado, L é o tamanho da posting list, f é o número de filtros, k é o número de itens de uma compra/reposição, H é o número de produtos distintos no histórico de um usuário, N é o tamanho do universo da entidade.

Tabela 1: Complexidade de tempo das operações e estruturas.

Operação	Tempo (médio)	Observação
Vetor: acesso []	$O(1)$	
Vetor: inserir ao final	$O(1)$ amortizado	dobramento de capacidade
Hash: inserir / buscar	$O(1)$	$O(L_{balde})$ no pior caso
AVL: inserir / buscar	$O(\log A)$	balanceada
AVL: faixa	$O(\log A + s)$	s = chaves no intervalo
U, P (cadastro)	$O(\log A)$	domínio limitado $\Rightarrow \approx O(1)$
R (reposição)	$O(k \log A + k^2)$	k itens (pequeno)
C (compra válida)	$O(k H)$	atualiza históricos
LU, LP (consulta)	$O(\sum_i L_i + R H)$	interseções + saída
LC, LR (consulta)	$O(\sum_i L_i + R k)$	posting lists diretas
NOT (complemento)	$O(N)$	varredura do universo

Detalhando as linhas principais, o cadastro insere o id em poucos índices, e como A é limitado pelo domínio cada inserção é praticamente constante. A reposição de k itens faz k inserções de timestamp e atualiza k posting lists. A compra válida verifica e atualiza

o estoque dos k produtos e ainda acumula os históricos do usuário e de cada produto, percorrendo listas de tamanho proporcional ao número de produtos distintos já comprados. As consultas LU e LP intersectam as f posting lists e imprimem os R resultados, cada um podendo exigir a leitura do histórico, de onde vem o termo de saída. A consulta por faixa custa $O(\log A + s)$, com s chaves no intervalo, e o complemento do NOT é o único que percorre todo o universo, $O(N)$, independentemente do tamanho do resultado.

Quanto ao **espaço**, as entidades ocupam $O(n + m + c + r)$. Os índices crescem proporcionalmente à soma dos tamanhos das posting lists, pois cada id aparece uma vez em cada atributo indexado. Como cada id ocupa poucos bytes, esse termo, ainda que linear em n , fica abaixo do espaço das entidades, dominado pelas cadeias de texto. Os históricos agregados acrescentam, no pior caso, um termo proporcional ao número de pares (usuário, produto) distintos efetivamente comprados. Não há estado redundante: o próximo id de cada entidade é o tamanho atual do vetor correspondente.

4 Estratégias de Robustez

Diversos mecanismos de programação defensiva foram incorporados:

- **Compra atômica:** o estoque de todos os produtos é verificado antes de qualquer alteração, uma compra inválida não modifica o estado, evitando estados parcialmente aplicados.
- **Verificação de limites:** consultas por id checam se o identificador é válido antes de acessar os vetores, prevenindo acessos indevidos à memória.
- **Preço em centavos:** o preço é indexado como inteiro (centavos), eliminando erros de comparação de ponto flutuante, a impressão usa duas casas decimais.
- **Regra dos três:** Vetor, Hash e AVL implementam construtor de cópia, operador de atribuição e destrutor, como aprendido em PDS2, prevenindo cópias rasas e dupla liberação na gerência manual de memória.
- **Leitura tolerante:** o tokenizador ignora espaços, tabulações e o caractere `\r` de finais de linha Windows, garantindo robustez na correção em Linux.

5 Análise Experimental

Metodologia. As medições usaram o *mesmo* código compilado com a macro `-DTP_STATS` (alvo `make stats`), que habilita a coleta de tempo por operação (com `<chrono>`) e de contadores estruturais (comparações, acessos, rotações), escritos apenas em `stderr`. Os cenários são determinísticos (seed fixa). A máquina usada foi MacBook com Apple M4 Pro, 24 GB de RAM, e linguagem C++11.

5.1 Perfis de carga

A Figura 1 compara três perfis: predominância de cadastros, de compras/reposições e de consultas. As operações de escrita são muito baratas (U entre 0,37 e 0,66 μs , C entre 0,63 e 1,10 μs), enquanto as consultas LU/LP custam de uma a três ordens de grandeza mais,

pois fazem interseção das posting lists e imprimem os históricos. Nota-se que LU/LP são mais caras no perfil de cadastros ($LP = 223,5 \mu s$) do que no de consultas ($LP = 63,1 \mu s$): o perfil de cadastros gera bases maiores, logo posting lists maiores e mais saída. As consultas LC/LR permanecem baratas ($< 3,4 \mu s$) por usarem índices diretos por id.

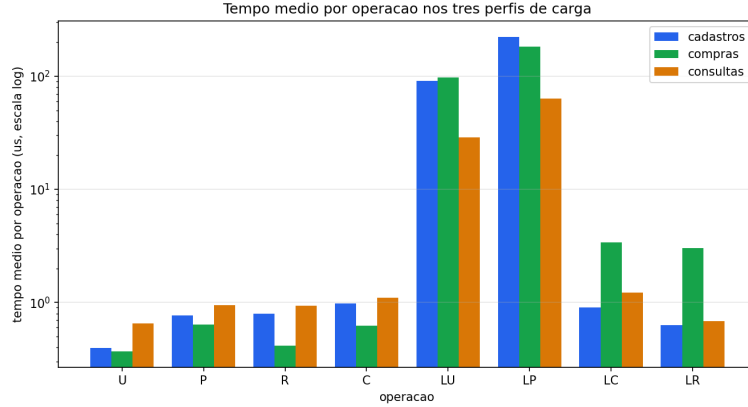


Figura 1: Tempo médio por operação nos três perfis (escala logarítmica).

Os contadores estruturais ajudam a entender o porquê. O perfil de compras/reposições é o que mais exercita a AVL (194.801 comparações e 14.970 rotações), pois cada **R** e **C** insere um timestamp na árvore. O perfil de consultas é o que mais acessa o **Vetor** (8,9 milhões de acessos), refletindo o percurso das posting lists nas interseções. Em todos os perfis a tabela hash não sofreu nenhum redimensionamento (`rehash = 0`): como os valores textuais distintos são poucos (há um número limitado de cidades, marcas etc.), a tabela nunca precisa crescer.

5.2 Escalabilidade

A Figura 2 mostra que o custo de *inserção* por índice é praticamente constante em n ($\approx 5,0$ comparações na AVL de idade e $\approx 4,0$ na hash, para n de mil a cinquenta mil). A razão é que a cardinalidade de valores distintos é limitada pelo domínio do atributo (idades de 18 a 70, conjunto fixo de cidades), e não por n . O crescimento com n não aparece na altura das árvores de índice, mas no tamanho das posting lists (append $O(1)$) e dos resultados das consultas, como confirma a Figura 3, em que o tempo de **LU** cresce de forma aproximadamente linear (dobra de $46,2 \mu s$ para $92,5 \mu s$ quando n passa de 10^4 para 2×10^4).

5.3 Número de filtros

A Figura 4 mostra um resultado que à primeira vista surpreende. O custo da consulta *cai* com mais filtros ($373,9 \mu s$ com um filtro contra $6,1 \mu s$ com quatro). A explicação é que o custo é dominado pelo tamanho do resultado e da saída, não pelo número de filtros em si. Um único filtro de cidade seleciona cerca de um oitavo dos usuários (resultado e impressão grandes), cada filtro adicional, combinado por interseção, poda fortemente o conjunto, reduzindo o trabalho e a saída.

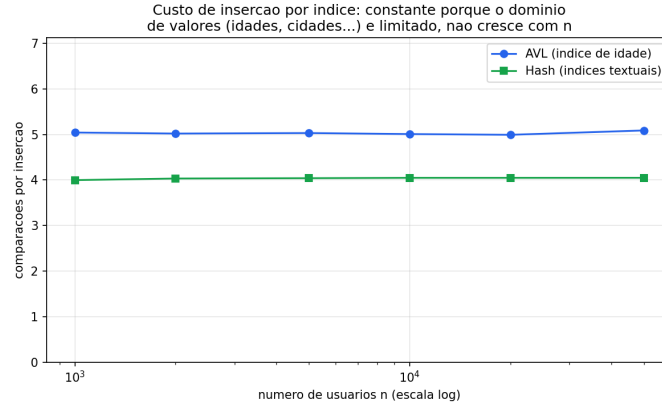


Figura 2: Custo de inserção por índice: constante porque o domínio de valores é limitado, não cresce com n .

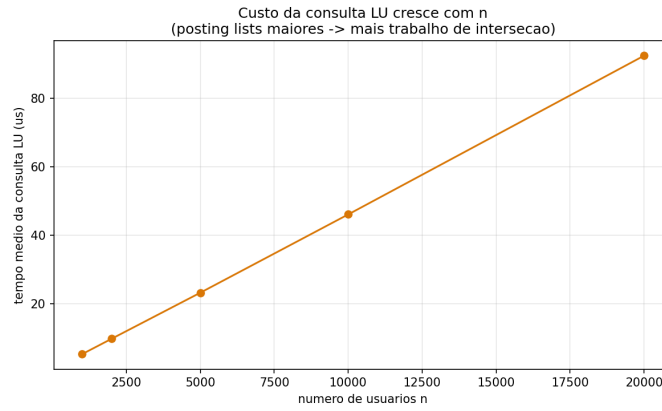


Figura 3: Tempo médio da consulta LU cresce de forma aproximadamente linear com n (posting lists maiores).

5.4 Comparação de estruturas: AVL vs. BST

O experimento que mais chama atenção está na Figura 5. Quando as chaves chegam em ordem crescente, que é o caso real do timestamp, uma BST simples degenera para $O(n^2)$. Para 10^5 chaves, ela faz cerca de 5 bilhões de comparações e leva $\approx 4,4$ s. A AVL, no mesmo cenário, faz 1,57 milhão de comparações e leva $\approx 7,5$ ms, quase três ordens de grandeza mais rápido. Com inserção aleatória a BST chega perto da AVL (2,0 contra 1,5 milhão de comparações), o que mostra que é justamente a entrada ordenada que a inviabiliza. É essa a razão de eu ter usado AVL nos índices numéricos.

5.5 Pontos extras

A consulta por **faixa** (Figura 6, esquerda) tem custo crescente com a largura do intervalo: o tempo e o número de nós visitados na AVL crescem juntos, de $205 \mu s$ e 10.304 visitas (largura 2) a $2863 \mu s$ e 64.022 visitas (largura 40), coerente com a complexidade $O(\log A + s)$, em que o termo s (chaves no intervalo) passa a dominar. Quanto às formas **booleanas** (Figura 6, direita), o AND é o mais barato ($58,9 \mu s$), pois a interseção poda o resultado, o OR é o mais caro ($834,8 \mu s$), pois a união amplia o resultado e a saída, o NOT ($371,0 \mu s$) reflete o custo $O(N)$ do complemento sobre o universo, seguido de poda pelo AND.

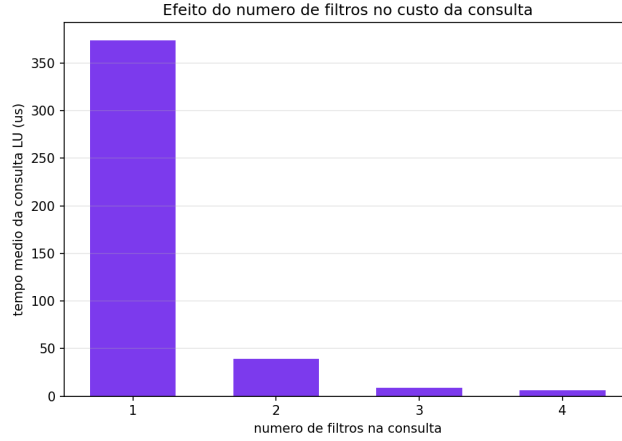


Figura 4: Efeito do número de filtros: filtros adicionais podam o resultado e *reduzem* o custo da consulta.

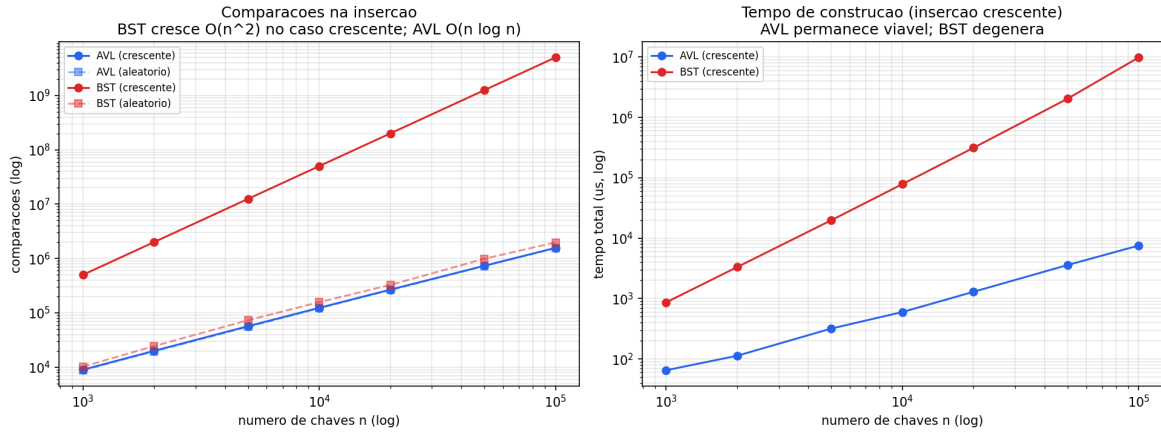


Figura 5: Inserção em ordem crescente: a BST cresce $O(n^2)$ enquanto a AVL permanece $O(n \log n)$, tanto em comparações quanto em tempo (escalas logarítmicas).

5.6 Uso de memória

O espaço total cresce de forma linear, $O(n + m + c + r)$, somando entidades, índices e históricos. Vale distinguir custo de tempo de custo de espaço: embora as posting lists dominem o tempo das consultas, por serem percorridas a cada interseção, em espaço elas são modestas, pois cada id ocupa poucos bytes. O termo dominante de memória são, na verdade, as próprias entidades, por causa das cadeias de texto. Um **Usuario**, por exemplo, guarda dois inteiros e quatro **strings**. Nos testes cada **string** ocupa 24 bytes, com buffer interno para textos curtos, o que dá cerca de 100 bytes por usuário antes de qualquer alocação no heap. Esse número depende da implementação, mas serve de ordem de grandeza.

Os índices acrescentam um termo menor. Cada id ocupa apenas 4 bytes e aparece uma vez por atributo indexado, então a presença de um usuário nos seus cinco índices soma cerca de 20 bytes nas posting lists, bem abaixo do custo da entidade. A tabela hash tem ainda um custo fixo por valor distinto, o nó com a chave e o ponteiro de encadeamento, mas como o número de valores distintos é limitado pelo domínio esse overhead não cresce com n , e em nenhum perfil houve redimensionamento. Por fim, os históricos agregados somam um termo proporcional ao número de pares (usuário, produto) efetivamente comprados.

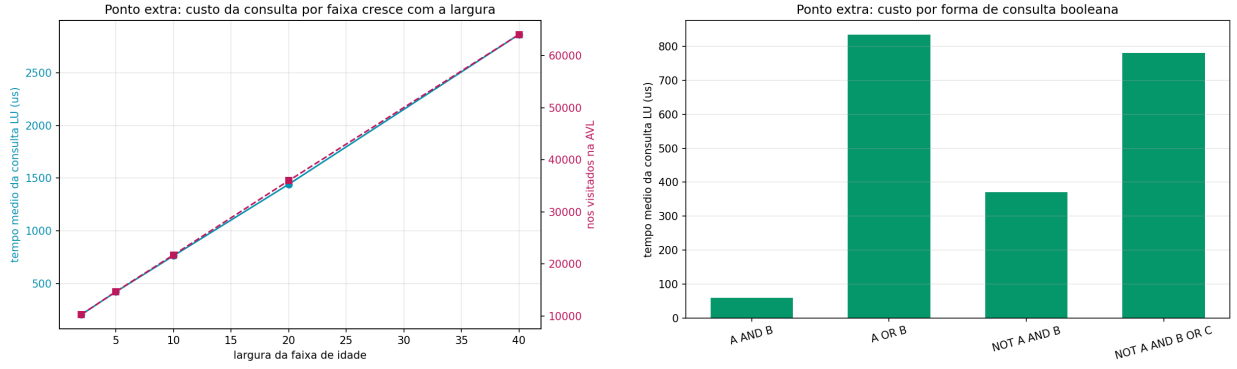


Figura 6: Pontos extras. Esquerda: custo da consulta por faixa cresce com a largura. Direita: custo por forma de consulta booleana (AND poda, OR amplia).

5.7 Localidade de referência

As posting lists e os vetores de entidades são contíguos em memória e percorridos sequencialmente nas interseções, o que favorece a localidade espacial e o aproveitamento de linhas de cache. Vale descrever o padrão de acesso das principais operações. Os cadastros (U, P) escrevem no final dos vetores e tocam poucos nós de índice, com acesso bem localizado. As compras e reposições (C, R) acessam um pequeno conjunto de produtos e inserem um nó de timestamp na AVL, num padrão mais disperso, porém limitado a k itens. As consultas (LU, LP, LC, LR) são as que mais varrem memória de forma sequencial, percorrendo as posting lists envolvidas, e é nelas que a contiguidade mais ajuda. O elevado número de acessos ao **Vetor** medido, milhões por execução, corresponde justamente a esses percursos. Uma lista encadeada teria a mesma complexidade assintótica, mas seria mais lenta na prática por espalhar os nós na memória. Por isso preferi o vetor contíguo.

6 Conclusões

Este trabalho implementou o MercadoDCC, um sistema de comércio eletrônico que mantém usuários, produtos, compras e reposições e responde a consultas por múltiplos atributos. A base da solução são índices invertidos apoiados em três estruturas próprias, um vetor dinâmico, uma tabela hash e uma árvore AVL, sem nenhum contêiner da biblioteca-padrão. Os dois pontos extras também foram feitos, os operadores booleanos com precedência e as consultas por faixa de valores numéricos.

A principal lição foi que a escolha da estrutura precisa acompanhar o padrão de acesso do problema, e não só a complexidade assintótica no papel. Indexar por valor de atributo, com os conjuntos de identificadores já ordenados, deixou as consultas multiatributo reduzidas a operações de conjunto. E manter tudo em vetores contíguos ajudou no tempo prático, por causa da localidade de referência.

Além disso, o balanceamento da AVL evitou a degeneração $O(n^2)$ que uma árvore comum teria com timestamps em ordem crescente, o que sozinho já justifica seu uso nos índices numéricos. Conclui também que o custo de manter os índices quase não cresce com n , porque o número de valores distintos é limitado pelo domínio do atributo. E que o tempo das consultas depende mais do tamanho do resultado do que do número de filtros, tanto que filtrar mais chegou a deixar a consulta mais rápida. No fim, o trabalho deixou claro como os índices invertidos e as estruturas de busca se encaixam para responder consultas por

vários atributos, além de que a escolha das estruturas de dados utilizadas e suas respectivas implementações importam muito em projetos de média e larga escala.

Referências

- [1] CORMEN, T. H. et al. *Introdução a Algoritmos*. 3^a ed. Elsevier, 2012.
- [2] MEIRA, W. *Implementação de Estrutura de Dados*. Slides da disciplina ED. DCC/UFGM.
- [3] AZPURUA, H. *Revisão de C++*. Slides da disciplina PDS2. DCC/UFGM.
- [4] Anthropic, Claude. Ferramenta auxiliar. Utilizado para indentação do código e geração de scripts de experimentos.